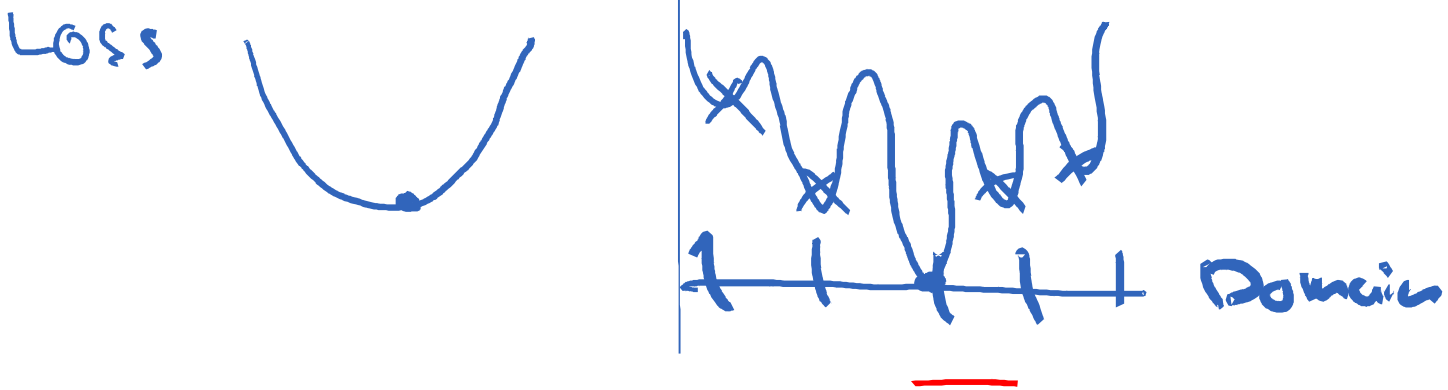
56262

Physics-informed Neural Networks 3

Self-adaptive physics-informed neural networks
(*Journal of Computational Physics, 2023*)

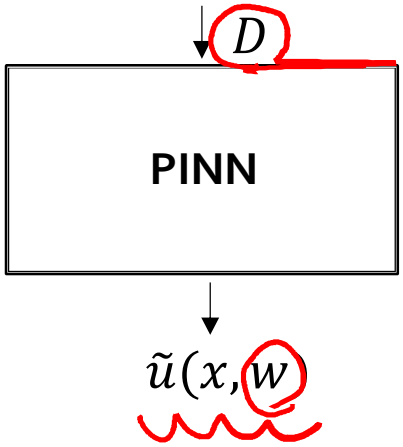
Problem of Vanilla PINN

Problem of Vanilla PINN	Alternatives	Keywords
Instance-wise retraining	Neural Operator	Operator learning
Noisy environments	Bayesian PINN	Uncertainty
Loss/gradient imbalance	**Self-adaptive PINN**	**Self-adaptive weighting**
Unstable training	X-PINN	Domain decomposition



PINN vs. SA-PINN 1/2

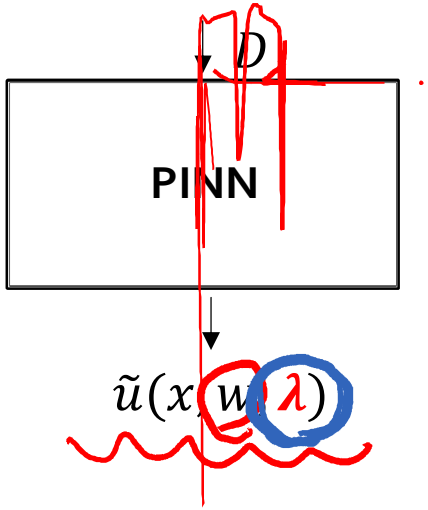
PINN



$$\mathcal{L}(\boldsymbol{w}) = \mathcal{L}_s(\boldsymbol{w}) + \mathcal{L}_r(\boldsymbol{w}) + \mathcal{L}_b(\boldsymbol{w}) + \mathcal{L}_0(\boldsymbol{w})$$

- Vanilla PINN treats all points almost uniformly,

Self Adaptive-PINN



$$\mathcal{L}(\boldsymbol{w}) = \lambda_s \mathcal{L}_s(\boldsymbol{w}) + \lambda_r \mathcal{L}_r(\boldsymbol{w}) + \lambda_b \mathcal{L}_b(\boldsymbol{w}) + \lambda_0 \mathcal{L}_0(\boldsymbol{w})$$

- SA-PINN emphasizes the difficult points more heavily.

PINN vs. SA-PINN 2/2

	Vanilla PINN	SA-PINN
Loss Weighting	Fixed or Manually tuned	Trainable, Pointowsie adaptive
Performance on stiff PDEs	Often unstable	More robust
Training strategy	Minimize loss	Min-maximize loss
Interpretation	Uniform fitting	Soft attention on difficult points

- **SA-PINN**
  - SA-PINNs extend standard PINNs by introducing trainable point-wise loss weights, enabling the network to automatically focus on difficult regions of the solution

Problem Statement

- **SA-PINN :** Difficulty-aware PDE solver
  - **Disadvantages of Vanilla PINNs :** Standard PINNs minimize several loss terms simultaneously, where some (easy) parts are learned much faster than others (hard).
  - **SA-PINNs** : address this issue by introducing adaptive weights at each individual training point, so that the model can automatically focus on hard-to-fit regions.
- **Core Idea**
  - Make the weights increase as the corresponding losses increase, forcing the neural network to pay more attention to difficult regions of solution.

Loss Function 1/2

- A standard PINN loss is written as

$$L(w) = L_s(w) + L_r(w) + L_b(w) + L_0(w)$$

NN parameter

$$\mathcal{L}_s(\boldsymbol{w}) = \frac{1}{2}\sum_{i=1}^{N_s} |u(\boldsymbol{x}_s^i, t_s^i; \boldsymbol{w}) - y_s^i|^2,$$

sample

$$\mathcal{L}_r(\boldsymbol{w}) = \frac{1}{2}\sum_{i=1}^{N_r} |\mathcal{N}_{\boldsymbol{x},t}[u(\boldsymbol{x}_r^i, t_r^i; \boldsymbol{w})] - f(\boldsymbol{x}_r^i, t_r^i)|^2,$$

physic

$$\mathcal{L}_b(\boldsymbol{w}) = \frac{1}{2}\sum_{i=1}^{N_b} |\mathcal{B}_{\boldsymbol{x},t}[u(\boldsymbol{x}_b^i, t_b^i; \boldsymbol{w})] - g(\boldsymbol{x}_b^i, t_b^i)|^2,$$

boundary

$$\mathcal{L}_0(\boldsymbol{w}) = \frac{1}{2}\sum_{i=1}^{N_0} |u(\boldsymbol{x}_0^i, 0; \boldsymbol{w}) - h(\boldsymbol{x}_0^i)|^2,$$

initial

- L_s: sensor/data loss,
- L_r: PDE residual loss,
- L_b: boundary condition loss,
- L_0: initial condition loss.

Loss Function 2/2

- SA-PINNs, trainable weights are introduced for each residual, boundary, and initial point:

$$\mathcal{L}(\boldsymbol{w}, \boldsymbol{\lambda}_r, \boldsymbol{\lambda}_b, \boldsymbol{\lambda}_0) = \mathcal{L}_s(\boldsymbol{w}) + \mathcal{L}_r(\boldsymbol{w}, \boldsymbol{\lambda}_r) + \mathcal{L}_b(\boldsymbol{w}, \boldsymbol{\lambda}_b) + \mathcal{L}_0(\boldsymbol{w}, \boldsymbol{\lambda}_0),$$

$$\mathcal{L}_r(\boldsymbol{w}, \boldsymbol{\lambda}_r) = \frac{1}{2} \sum_{i=1}^{N_r} m(\lambda_r^i) |\mathcal{N}_{\boldsymbol{x},t}[u(\boldsymbol{x}_r^i, t_r^i; \boldsymbol{w})] - f(\boldsymbol{x}_r^i, t_r^i)|^2$$

$$\mathcal{L}_b(\boldsymbol{w}, \boldsymbol{\lambda}_b) = \frac{1}{2} \sum_{i=1}^{N_b} m(\lambda_b^i) |\mathcal{B}_{\boldsymbol{x},t}[u(\boldsymbol{x}_r^i, t_r^i; \boldsymbol{w})] - g(\boldsymbol{x}_b^i, t_b^i)|^2$$

$$\mathcal{L}_0(\boldsymbol{w}, \boldsymbol{\lambda}_0) = \frac{1}{2} \sum_{i=1}^{N_0} m(\lambda_0^i) |u(\boldsymbol{x}_0^i, 0; \boldsymbol{w}) - h(\boldsymbol{x}_0^i)|^2,$$

- Self-adaptation mask function $m(\lambda)$:
  1. Nonnegative, differentiable, strictly increasing function of λ.

Minimax Weighting Schem

- **Saddle point optimization**
  - The loss $L(w, \lambda_r, \lambda_i, \lambda_0)$ is minimized with respect to the network weights w, as usual,
  - But is also maximized with respect to the self-adaptation weights $\lambda_r, \lambda_i, \lambda_0$
- **The corresponding gradient descent/ascent steps are:**

Minimized

$$\boldsymbol{w}^{k+1} = \boldsymbol{w}^k - \eta_k \nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}^k, \boldsymbol{\lambda}_r^k, \boldsymbol{\lambda}_b^k, \boldsymbol{\lambda}_0^k)$$

Maximized

$$\boldsymbol{\lambda}_r^{k+1} = \boldsymbol{\lambda}_r^k + \rho_r^k \nabla_{\boldsymbol{\lambda}_r} \mathcal{L}(\boldsymbol{w}^k, \boldsymbol{\lambda}_r^k, \boldsymbol{\lambda}_b^k, \boldsymbol{\lambda}_0^k)$$

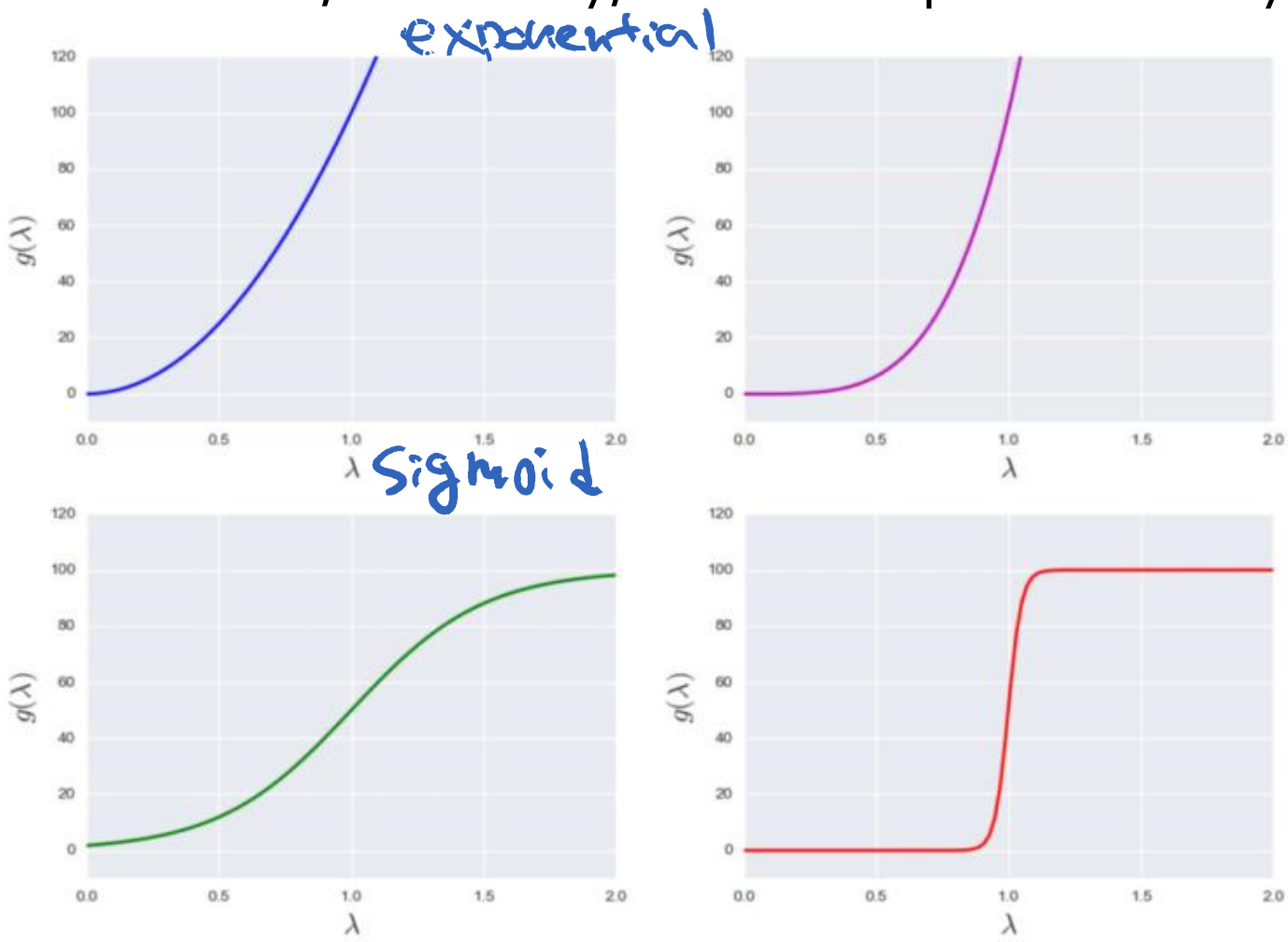
$$\boldsymbol{\lambda}_b^{k+1} = \boldsymbol{\lambda}_b^k + \rho_b^k \nabla_{\boldsymbol{\lambda}_b} \mathcal{L}(\boldsymbol{w}^k, \boldsymbol{\lambda}_r^k, \boldsymbol{\lambda}_b^k, \boldsymbol{\lambda}_0^k)$$

$$\boldsymbol{\lambda}_0^{k+1} = \boldsymbol{\lambda}_0^k + \rho_0^k \nabla_{\boldsymbol{\lambda}_0} \mathcal{L}(\boldsymbol{w}^k, \boldsymbol{\lambda}_r^k, \boldsymbol{\lambda}_b^k, \boldsymbol{\lambda}_0^k),$$

The network minimizes the losses while the adaptive weights increase the penalty on difficult points

Self-adaptation Mask Function

- $m(\lambda)$ progressively penalizes the network more for not fitting the residual, boundary, and initial points closely.



Saddle Point Optimization 1/3

- **Saddle point optimization**
  - Although the adaptive weights $m(\lambda)$ are monotonically increasing, this does not mean that training becomes divergent.
  - The reason is that the growth of λ is coupled to the error at each training point, and that error is simultaneously being reduced by the neural network parameters w.

$$L_i(w, \lambda_i) = \frac{1}{2} m(\lambda_i) e_i(w)^2,$$

- $e_i(w)$ is the unmasked error at that point,
- $m(\lambda_i)$ is the adaptive mask,
- and $m'(\lambda_i) > 0$ because the mask function is strictly increasing.

Saddle Point Optimization 2/3

- **Saddle point optimization**

$$L_i(w, \lambda_i) = \frac{1}{2} m(\lambda_i) e_i(w)^2,$$

  - Differentiate with respect to λ_i:

$$\frac{\partial L_i}{\partial \lambda_i} = \frac{1}{2} m'(\lambda_i) e_i(w)^2.$$

  - If the error is large, the weight grows faster

$$\lambda_i^{k+1} = \lambda_i^k + \rho \frac{1}{2} m'(\lambda_i^k) e_i(w^k)^2,$$

$$e_i \uparrow \Rightarrow \lambda_i \uparrow \Rightarrow m(\lambda_i) \uparrow \Rightarrow \text{stronger correction on } w \Rightarrow e_i \downarrow .$$

Saddle Point Optimization 3/3

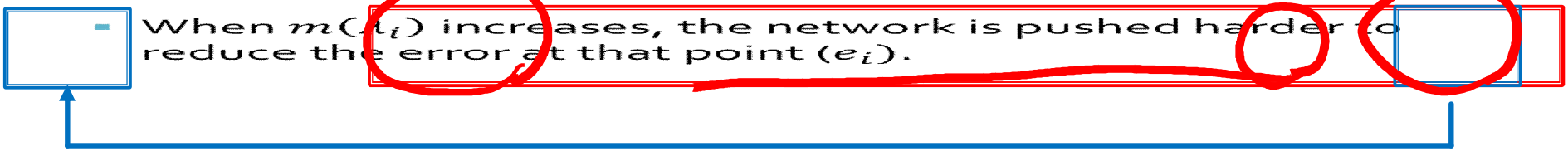
- **Saddle point optimization**

$$L_i(w, \lambda_i) = \frac{1}{2} m(\lambda_i) e_i(w)^2,$$

  - Differentiate with respect to the same point i:

$$\nabla_w L_i = m(\lambda_i) e_i(w) \nabla_w e_i(w).$$

  - When $m(\lambda_i)$ increases, the network is pushed harder to reduce the error at that point (e_i).



Numerical Examples 1/3

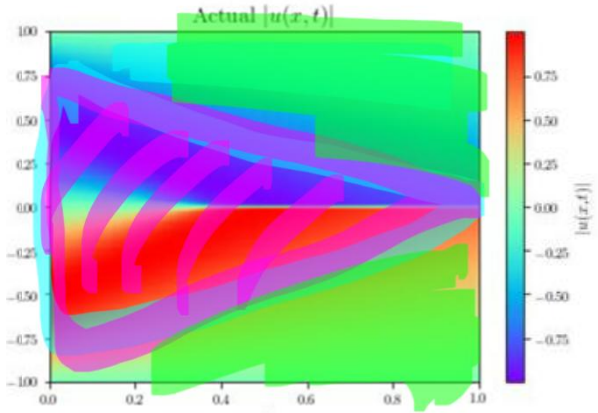
- **Viscous Burgers Equation** (Fluid Dynamics)

$$u_t + uu_x - (0.01/\pi)u_{xx} = 0, \quad x \in [-1, 1], \; t \in [0, 1],$$

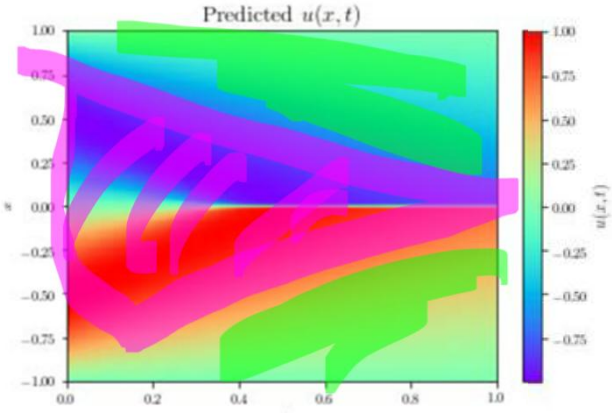
$$u(0, x) = -\sin(\pi x), \qquad \text{Initial condition}$$

$$u(t, -1) = u(t, 1) = 0. \qquad \text{Boundary condition}$$

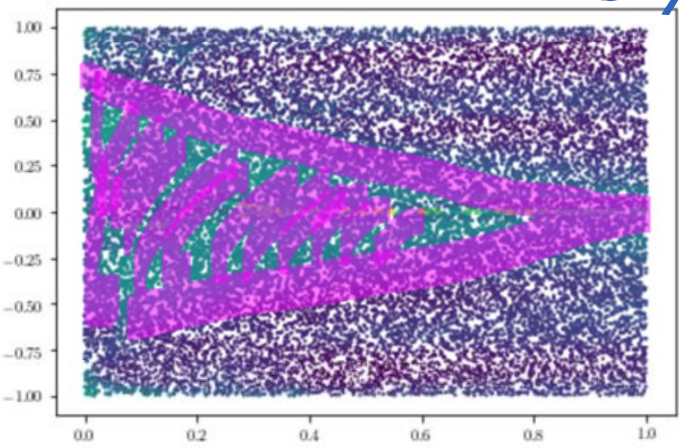
u GT



u Estimated



Trained Weights (x)



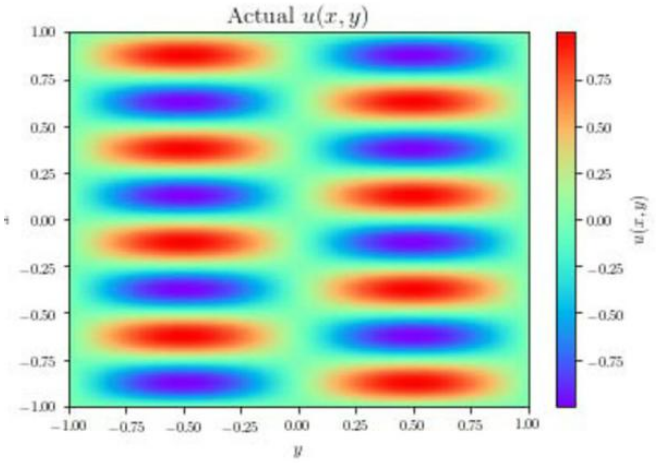
Numerical Examples 2/3

- **Helmholtz Equation** (Wave and Diffusion processes)

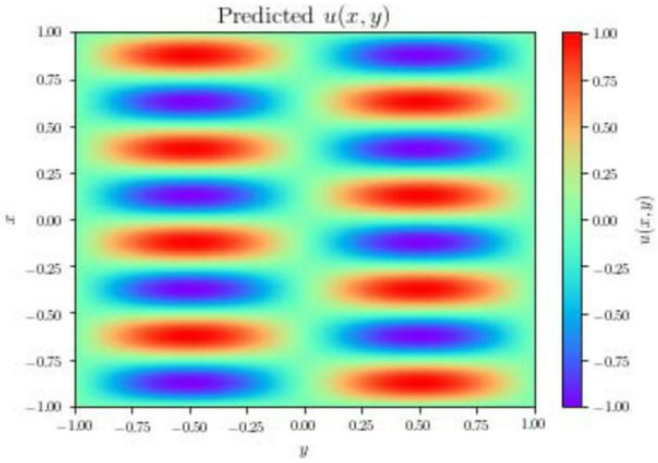
$$u_{xx} + u_{yy} + k^2 u - q(x, y) = 0$$

$$u(-1, y) = u(1, y) = u(x, -1) = u(x, 1) = 0$$ Boundary condition

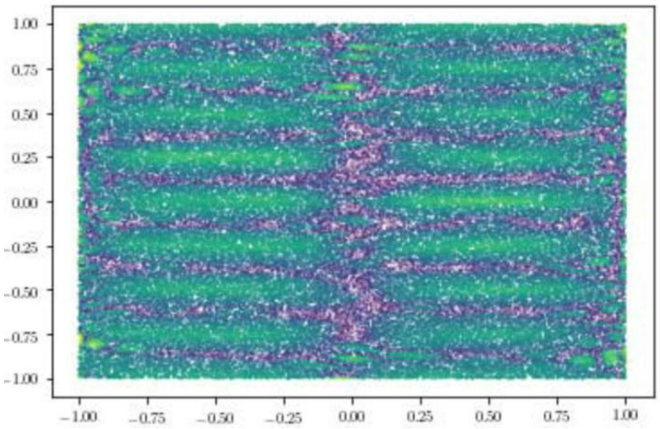
u GT



u Estimated



Trained Weights λ



Numerical Examples 3/3

- **Allen-Cahn Equation** (Phase-field models)

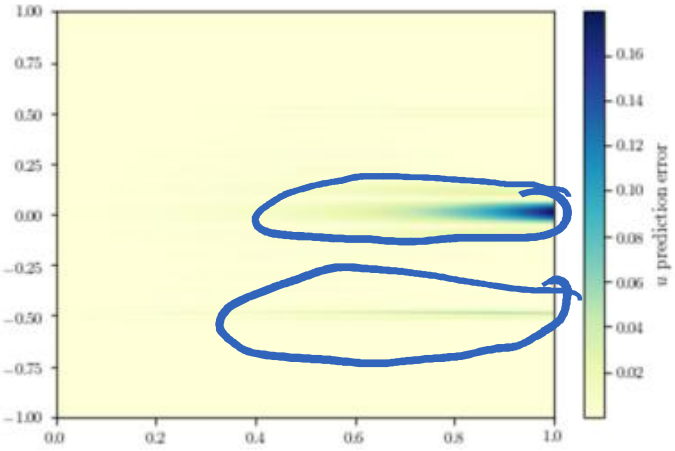
$$u_t - 0.0001u_{xx} + 5u^3 - 5u = 0, \quad x \in [-1, 1], \; t \in [0, 1],$$

$$u(x, 0) = x^2 cos(\pi x), \quad \text{Initial condition}$$

$$u(t, -1) = u(t, 1), \quad \text{Boundary condition}$$

Notice that the initial condition weights grow much faster than the residue weights, as expected, since the initial condition tends to be neglected by the PINN,

Errors



Trained Weights (λ)

